

PHP تصميم نظام متجر إلكتروني بلغة

1. هيكل الفئات

```
php
<?php

// =====
// فئة المنتج (Product)
// =====
class Product {
    // الخصائص الخاصة (Private)
    private string $id;
    private string $name;
    private float $price;
    private int $quantity;
    private float $discountPercentage;
    private DateTime $createdAt;
    private bool $isActive;

    // البناء (Constructor)
    public function __construct(
        string $name,
        float $price,
        int $quantity = 0,
        float $discountPercentage = 0.0
    ) {
        $this->id = uniqid('prod_', true);
        $this->name = $name;
        $this->setPrice($price); // للتحقق استخدام setter
        $this->setQuantity($quantity);
        $this->setDiscountPercentage($discountPercentage);
        $this->createdAt = new DateTime();
        $this->isActive = true;
    }
}
```

```

// ===== Getters =====
public function getId(): string {
    return $this->id;
}

public function getName(): string {
    return $this->name;
}

public function getPrice(): float {
    return $this->price;
}

public function getQuantity(): int {
    return $this->quantity;
}

public function getDiscountPercentage(): float {
    return $this->discountPercentage;
}

public function getCreatedAt(): DateTime {
    return $this->createdAt;
}

public function isActive(): bool {
    return $this->isActive;
}

// ===== Setters مع التحقق =====
public function setName(string $name): void {
    if (strlen(trim($name)) < 2) {
        throw new InvalidArgumentException("اسم المنتج يجب أن يكون  

        "ن على الأقل حرفين");
    }
    $this->name = $name;
}

public function setPrice(float $price): void {

```

```

        if ($price < 0) {
            throw new InvalidArgumentException("السعر لا يمكن أن يكون سالباً");
        }
        if ($price > 1000000) {
            throw new InvalidArgumentException("السعر كبير جداً");
        }
        $this->price = $price;
    }

    public function setQuantity(int $quantity): void {
        if ($quantity < 0) {
            throw new InvalidArgumentException("الكمية لا يمكن أن تكون سالبة");
        }
        $this->quantity = $quantity;
    }

    public function setDiscountPercentage(float $discount): void {
        if ($discount < 0 || $discount > 100) {
            throw new InvalidArgumentException("نسبة الخصم يجب أن تكون بين 0 و 100");
        }
        $this->discountPercentage = $discount;
    }

    public function deactivate(): void {
        $this->isActive = false;
    }

    public function activate(): void {
        $this->isActive = true;
    }

    // ===== الدوال العامة =====
    public function calculatePriceAfterDiscount(): float {
        $discountAmount = $this->price * ($this->discountPercentage / 100);
        return $this->price - $discountAmount;
    }

```

```

    }

    public function increaseQuantity(int $amount): void {
        if ($amount <= 0) {
            throw new InvalidArgumentException("القيمة يجب أن تكون موجبة");
        }
        $this->quantity += $amount;
    }

    public function decreaseQuantity(int $amount): void {
        if ($amount <= 0) {
            throw new InvalidArgumentException("القيمة يجب أن تكون موجبة");
        }
        if ($this->quantity - $amount < 0) {
            throw new InvalidArgumentException("الكمية غير كافية");
        }
        $this->quantity -= $amount;
    }

    public function hasStock(): bool {
        return $this->quantity > 0;
    }

    public function hasStockFor(int $required): bool {
        return $this->quantity >= $required;
    }

    // ===== الدوال السحرية =====
    public function __toString(): string {
        return sprintf(
            "المنتج: %s (السعر: 2.%, الكمية: %d, الخصم: 1.%f%%)",
            $this->name,
            $this->price,
            $this->quantity,
            $this->discountPercentage
        );
    }
}

```

```

public function __get(string $property) {
    if (property_exists($this, $property)) {
        return $this->$property;
    }
    throw new Exception("الخاصية {$property} غير موجودة");
}

public function __set(string $property, $value) {
    // لا تسمح بالتعديل المباشر للأسعار
    if ($property === 'price') {
        throw new Exception("لا يمكن تعديل السعر مباشرة. استخدم setPrice()");
    }

    if (property_exists($this, $property)) {
        $setter = 'set' . ucfirst($property);
        if (method_exists($this, $setter)) {
            $this->$setter($value);
        } else {
            $this->$property = $value;
        }
    }
}

public function __debugInfo(): array {
    return [
        'id' => $this->id,
        'name' => $this->name,
        'price' => $this->price,
        'quantity' => $this->quantity,
        'discount' => $this->discountPercentage . '%',
        'active' => $this->isActive ? 'نشط' : 'غير نشط'
    ];
}
}

// =====
// فئة العميل (Customer)

```

```
// =====
class Customer {
    // الخصائص الخاصة
    private string $id;
    private string $name;
    private string $email;
    private DateTime $registrationDate;
    private string $phone;
    private array $addresses;
    private bool $isVerified;
    private array $orderHistory;

    // البناء (Constructor)
    public function __construct(
        string $name,
        string $email,
        string $phone = ''
    ) {
        $this->id = uniqid('cust_', true);
        $this->setName($name);
        $this->setEmail($email);
        $this->setPhone($phone);
        $this->registrationDate = new DateTime();
        $this->addresses = [];
        $this->isVerified = false;
        $this->orderHistory = [];
    }

    // ===== Getters =====
    public function getId(): string {
        return $this->id;
    }

    public function getName(): string {
        return $this->name;
    }

    public function getEmail(): string {
        return $this->email;
    }
}
```

```

    }

    public function getRegistrationDate(): DateTime {
        return $this->registrationDate;
    }

    public function getPhone(): string {
        return $this->phone;
    }

    public function getAddresses(): array {
        return $this->addresses;
    }

    public function isVerified(): bool {
        return $this->isVerified;
    }

    // ===== Setters مع التحقق =====
    public function setName(string $name): void {
        if (strlen(trim($name)) < 2) {
            throw new InvalidArgumentException("الاسم يجب أن يكون على الأقل حرفين");
        }
        $this->name = $name;
    }

    public function setEmail(string $email): void {
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException("البريد الإلكتروني غير صالح");
        }
        $this->email = $email;
    }

    public function setPhone(string $phone): void {
        // تحقق بسيط من رقم الهاتف
        if (!empty($phone) && !preg_match('/^[0-9+\\-\\s]{10,20}$/', $phone)) {

```

```

        throw new InvalidArgumentException("رقم الهاتف غير صالح");
    }
    $this->phone = $phone;
}

public function verify(): void {
    $this->isVerified = true;
}

// ===== الدوال العامة =====
public function calculateMembershipAge(): DateInterval {
    $now = new DateTime();
    return $this->registrationDate->diff($now);
}

public function getMembershipAgeInDays(): int {
    $interval = $this->calculateMembershipAge();
    return (int)$interval->format('%a');
}

public function getMembershipAgeFormatted(): string {
    $interval = $this->calculateMembershipAge();

    if ($interval->y > 0) {
        return "{$interval->y} سنة و {$interval->m} شهر";
    } elseif ($interval->m > 0) {
        return "{$interval->m} شهر و {$interval->d} يوم";
    } else {
        return "{$interval->d} يوم";
    }
}

public function addAddress(array $address): void {
    $required = ['street', 'city', 'country'];
    foreach ($required as $field) {
        if (!isset($address[$field])) {
            throw new InvalidArgumentException("مطلوب {$field} حقول");
        }
    }
}

```

```

    }
    $this->addresses[] = $address;
}

public function addToOrderHistory(Order $order): void {
    $this->orderHistory[] = [
        'order_id' => $order->getId(),
        'date' => $order->getOrderDate()->format('Y-m-d'),
        'total' => $order->calculateTotal()
    ];
}

// ===== الدوال السحرية =====
public function __toString(): string {
    return sprintf(
        "العضوية منذ : %s (البريد : %s, العميل : %s)",
        $this->name,
        $this->email,
        $this->getMembershipAgeFormatted()
    );
}

public function __call(string $method, array $args) {
    if (strpos($method, 'get') === 0) {
        $property = lcfirst(substr($method, 3));
        if (property_exists($this, $property)) {
            return $this->$property;
        }
    }
    throw new Exception("الدالة {$method} غير موجودة");
}
}

// =====
// فئة الطلب (Order)
// =====
class Order {
    // الخصائص الخاصة
    private string $id;

```

```

private DateTime $orderDate;
private string $status;
private Customer $customer;
private array $items; // مصفوفة من المنتجات والكميات
private float $shippingCost;
private float $taxRate;
private string $notes;

// ثوابت الحالة
public const STATUS_PENDING = 'pending';
public const STATUS_PROCESSING = 'processing';
public const STATUS_SHIPPED = 'shipped';
public const STATUS_DELIVERED = 'delivered';
public const STATUS_CANCELLED = 'cancelled';

// البناء (Constructor)
public function __construct(Customer $customer, float $shippingCost
= 0.0) {
    $this->id = 'ORD_' . date('Ymd') . '_' . uniqid();
    $this->orderDate = new DateTime();
    $this->status = self::STATUS_PENDING;
    $this->customer = $customer;
    $this->items = [];
    $this->setShippingCost($shippingCost);
    $this->taxRate = 0.15; // 15% ضريبة افتراضية
    $this->notes = '';
}

// ===== Getters =====
public function getId(): string {
    return $this->id;
}

public function getOrderDate(): DateTime {
    return $this->orderDate;
}

public function getStatus(): string {
    return $this->status;
}

```

```

    }

    public function getCustomer(): Customer {
        return $this->customer;
    }

    public function getItems(): array {
        return $this->items;
    }

    public function getShippingCost(): float {
        return $this->shippingCost;
    }

    public function getNotes(): string {
        return $this->notes;
    }

    // ===== Setters مع التحقق =====
    public function setStatus(string $status): void {
        $validStatuses = [
            self::STATUS_PENDING,
            self::STATUS_PROCESSING,
            self::STATUS_SHIPPED,
            self::STATUS_DELIVERED,
            self::STATUS_CANCELLED
        ];

        if (!in_array($status, $validStatuses)) {
            throw new InvalidArgumentException("حالة الطلب غير صالحة");
        }
        $this->status = $status;
    }

    public function setShippingCost(float $cost): void {
        if ($cost < 0) {
            throw new InvalidArgumentException("تكلفة الشحن لا يمكن أن تكون سالبة");
        }
    }

```

```

    }
    $this->shippingCost = $cost;
}

public function setNotes(string $notes): void {
    $this->notes = $notes;
}

// ===== إدارة المنتجات =====
public function addItem(Product $product, int $quantity = 1): void {
    if ($quantity <= 0) {
        throw new InvalidArgumentException("الكمية يجب أن تكون أكبر من الصفر");
    }

    if (!$product->hasStockFor($quantity)) {
        throw new Exception("الكمية غير متوفرة في المخزون");
    }

    // تقليل كمية المنتج في المخزون
    $product->decreaseQuantity($quantity);

    // إضافة المنتج للطلب
    $this->items[] = [
        'product' => $product,
        'quantity' => $quantity,
        'unit_price' => $product->calculatePriceAfterDiscount(),
        'added_at' => new DateTime()
    ];
}

public function removeItem(int $index): void {
    if (!isset($this->items[$index])) {
        throw new InvalidArgumentException("عنصر الطلب غير موجود");
    }
}

// إرجاع الكمية للمخزون
$item = $this->items[$index];

```

```

$item['product']->increaseQuantity($item['quantity']);

unset($this->items[$index]);
$this->items = array_values($this->items); // إعادة ترتيب المص
فوفة
}

public function updateItemQuantity(int $index, int $newQuantity): void
id {
    if (!isset($this->items[$index])) {
        throw new InvalidArgumentException("عنصر الطلب غير موجود");
    ;

    }

    if ($newQuantity <= 0) {
        $this->removeItem($index);
        return;
    }

    $item = $this->items[$index];
    $oldQuantity = $item['quantity'];
    $product = $item['product'];

    // حساب الفرق وإعادة ضبط المخزون
    $difference = $newQuantity - $oldQuantity;

    if ($difference > 0) {
        // زيادة الكمية
        if (!$product->hasStockFor($difference)) {
            throw new Exception("الكمية الإضافية غير متوفرة في المخزون");
        }
        $product->decreaseQuantity($difference);
    } else {
        // تقليل الكمية
        $product->increaseQuantity(abs($difference));
    }

    $this->items[$index]['quantity'] = $newQuantity;

```

```

}

// ===== الحسابات المالية =====
public function calculateSubtotal(): float {
    $subtotal = 0.0;

    foreach ($this->items as $item) {
        $subtotal += $item['unit_price'] * $item['quantity'];
    }

    return $subtotal;
}

public function calculateTax(): float {
    return $this->calculateSubtotal() * $this->taxRate;
}

public function calculateTotal(): float {
    $subtotal = $this->calculateSubtotal();
    $tax = $this->calculateTax();

    return $subtotal + $tax + $this->shippingCost;
}

public function getTotalDiscount(): float {
    $totalDiscount = 0.0;

    foreach ($this->items as $item) {
        $product = $item['product'];
        $originalPrice = $product->getPrice();
        $discountedPrice = $product->calculatePriceAfterDiscount();
        $discountPerUnit = $originalPrice - $discountedPrice;

        $totalDiscount += $discountPerUnit * $item['quantity'];
    }

    return $totalDiscount;
}

```

```
// ===== دوال مساعدة =====
public function getItemCount(): int {
    return count($this->items);
}

public function getTotalQuantity(): int {
    $total = 0;
    foreach ($this->items as $item) {
        $total += $item['quantity'];
    }
    return $total;
}

public function cancel(): void {
    if ($this->status === self::STATUS_DELIVERED) {
        throw new Exception("لا يمكن إلغاء طلب تم توصيله");
    }

    // إرجاع جميع المنتجات للمخزون
    foreach ($this->items as $item) {
        $item['product']->increaseQuantity($item['quantity']);
    }

    $this->setStatus(self::STATUS_CANCELLED);
}

// ===== الدوال السحرية =====
public function __toString(): string {
    return sprintf(
        "الإجمالي: %2s, التاريخ: %s, الحالة: %s (الطلب: #%s)",
        $this->id,
        $this->getStatusText(),
        $this->orderDate->format('Y-m-d'),
        $this->calculateTotal()
    );
}

public function __sleep(): array {
    // serialize تحديد الخصائص التي سيتم تخزينها عند
```

```

        return ['id', 'orderDate', 'status', 'items', 'shippingCost', 'taxRate', 'notes'];
    }

    public function __wakeup(): void {
        // إعادة تهيئة بعد unserialize
        if (!isset($this->orderDate)) {
            $this->orderDate = new DateTime();
        }
    }

    public function __invoke(): array {
        // عند استدعاء الكائن كدالة
        return [
            'id' => $this->id,
            'date' => $this->orderDate->format('Y-m-d H:i:s'),
            'status' => $this->status,
            'customer' => $this->customer->getName(),
            'item_count' => $this->getItemCount(),
            'total' => $this->calculateTotal()
        ];
    }

    // ===== دوال إضافية =====
    private function getStatusText(): string {
        $statusMap = [
            self::STATUS_PENDING => 'قيد الانتظار',
            self::STATUS_PROCESSING => 'قيد المعالجة',
            self::STATUS_SHIPPED => 'تم الشحن',
            self::STATUS_DELIVERED => 'تم التوصيل',
            self::STATUS_CANCELLED => 'ملغي'
        ];

        return $statusMap[$this->status] ?? $this->status;
    }
}

// =====
// فئة نظام المتجر (ECommerceSystem)

```

```
// =====
class ECommerceSystem {
    private array $products = [];
    private array $customers = [];
    private array $orders = [];
    private float $totalRevenue = 0.0;

    public function addProduct(Product $product): void {
        $this->products[$product->getId()] = $product;
    }

    public function addCustomer(Customer $customer): void {
        $this->customers[$customer->getId()] = $customer;
    }

    public function createOrder(Customer $customer): Order {
        $order = new Order($customer);
        $this->orders[$order->getId()] = $order;
        return $order;
    }

    public function getProduct(string $id): ?Product {
        return $this->products[$id] ?? null;
    }

    public function getCustomer(string $id): ?Customer {
        return $this->customers[$id] ?? null;
    }

    public function getOrder(string $id): ?Order {
        return $this->orders[$id] ?? null;
    }

    public function getLowStockProducts(int $threshold = 10): array {
        $lowStock = [];

        foreach ($this->products as $product) {
            if ($product->getQuantity() <= $threshold && $product->isActive()) {

```

```

        $lowStock[] = $product;
    }
}

return $lowStock;
}

public function generateSalesReport(DateTime $startDate, DateTime $endDate): array {
    $report = [
        'period' => $startDate->format('Y-m-d') . ' إلى ' . $endDate->format('Y-m-d'),
        'total_orders' => 0,
        'total_revenue' => 0.0,
        'total_items_sold' => 0,
        'orders' => []
    ];

    foreach ($this->orders as $order) {
        $orderDate = $order->getOrderDate();

        if ($orderDate >= $startDate && $orderDate <= $endDate) {
            if ($order->getStatus() !== Order::STATUS_CANCELLED) {
                $report['total_orders']++;
                $report['total_revenue'] += $order->calculateTotal();
            }

            $report['total_items_sold'] += $order->getTotalQuantity();

            $report['orders'][] = $order->getId();
        }
    }

    return $report;
}

// =====
// مثال على استخدام النظام

```

```
// =====
function runExample() {
    echo "<h2>مثال عملي لنظام المتجر الإلكتروني</h2>\n";

    // إنشاء نظام المتجر 1.
    $system = new ECommerceSystem();

    // إضافة منتجات 2.
    $product1 = new Product('لابتوب ديل', 3500.00, 50, 10.0);
    $product2 = new Product('ماوس لاسلكي', 150.00, 100, 5.0);
    $product3 = new Product('لوحة مفاتيح ميكانيكية', 300.00, 30, 15.0);
;

    $system->addProduct($product1);
    $system->addProduct($product2);
    $system->addProduct($product3);

    echo "<h3>المنتجات:</h3>\n";
    echo $product1 . "<br>\n";
    echo $product2 . "<br>\n";
    echo $product3 . "<br>\n";

    // إضافة عميل 3.
    $customer = new Customer('أحمد محمد', 'ahmed@example.com', '+966501
234567');
    $customer->verify();
    $customer->addAddress([
        'street' => 'شارع الملك فهد',
        'city' => 'الرياض',
        'country' => 'السعودية',
        'postal_code' => '12345'
    ]);

    $system->addCustomer($customer);

    echo "<h3>العميل:</h3>\n";
    echo $customer . "<br>\n";
    echo "عمر العضوية: " . $customer->getMembershipAgeFormatted() . "<br>\n";
}
```

```

// إنشاء طلب 4.
$order = $system->createOrder($customer);
$order->addItem($product1, 2);
$order->addItem($product2, 1);
$order->setShippingCost(50.00);
$order->setNotes('التوصيل خلال 3 أيام عمل');
$order->setStatus(Order::STATUS_PROCESSING);

echo "<h3>الطلب:</h3>\n";
echo $order . "<br>\n";
echo "المجموع الجزئي: " . $order->calculateSubtotal() . " ريال<br>\n"
;

echo "الضريبة: " . $order->calculateTax() . " ريال<br>\n";
echo "الشحن: " . $order->getShippingCost() . " ريال<br>\n";
echo "الإجمالي: " . $order->calculateTotal() . " ريال<br>\n";
echo "إجمالي الخصم: " . $order->getTotalDiscount() . " ريال<br>\n";

// استخدام الدوال السحرية 5.
echo "<h3>استخدام الدوال السحرية:</h3>\n";

// __toString
echo "عرض المنتج: " . $product1 . "<br>\n";

// __debugInfo (استخدام var_dump)
echo "<pre>";
var_dump($product1);
echo "</pre>";

// __get و __set
try {
    // محاولة تعديل السعر مباشرة (سيتم رفضها)
    // $product1->price = 4000; // سيثير استثناء
    $product1->setPrice(4000); // الطريقة الصحيحة
    echo "<br>\nتم تعديل سعر المنتج بنجاح";
} catch (Exception $e) {
    echo "خطأ: " . $e->getMessage() . "<br>\n";
}

```

```

// __invoke
echo "<pre>";
print_r($order());
echo "</pre>";

// تقرير المبيعات 6.
$startDate = new DateTime('2024-01-01');
$endDate = new DateTime('2024-12-31');
$report = $system->generateSalesReport($startDate, $endDate);

echo "<h3>تقرير المبيعات:</h3>\n";
echo "<pre>";
print_r($report);
echo "</pre>";
}

// تشغيل المثال
runExample();

?>

```

إجابات الأسئلة:

1. كيف تصمم هيكل الفئات لهذا النظام؟

- تمثل المنتجات مع خصائص (الاسم، السعر، الكمية، الخصم) فئة **Product**
- تمثل العملاء مع خصائص (الاسم، البريد، تاريخ التسجيل) فئة **Customer**
- تمثل الطلبات مع خصائص (رقم الطلب، التاريخ، الحالة، العميل، العناصر) فئة **Order**
- لإدارة النظام ككل فئة **ECommerceSystem**

2. public وما هي private ما هي الخصائص التي سأجعلها؟

- **Private:** جميع الخصائص الأساسية (id, name, price, quantity, etc.)
- **Public:** فقط للوصول والتعديل مع التحقق (methods) الدوال
- لمنع التعديل المباشر (Encapsulation) إخفاء البيانات: التبرير

3. في كل فئة؟ (Constructor) كيف ستخدم البناء.

- يأخذ (الاسم، السعر، الكمية، نسبة الخصم) مع قيم افتراضية: **Product**
- يأخذ (الاسم، البريد، الهاتف) مع تهيئة تاريخ التسجيل: **Customer**
- يأخذ (العميل، تكلفة الشحن) مع إنشاء رقم طلب فريد: **Order**

4. ما هي الدوال السحرية التي يمكن إضافتها؟

- لتمثيل الكائن كنص: **__toString()**
- للتحكم في الوصول للخصائص: **__set()** و **__get()**
- لعرض معلومات مصغرة عند التصحيح: **__debugInfo()**
- للتخزين والإستعادة: **__wakeup()** و **__sleep()**
- لاستدعاء الكائن كدالة: **__invoke()**

5. كيف تحمي بيانات الأسعار من التعديل المباشر؟

- **private** جعل الخاصية: `private float $price;`
- مع التحقق **setter** توفير:

php

```
public function setPrice(float $price): void {  
    if ($price < 0) {  
        throw new InvalidArgumentException("السعر لا يمكن أن يكون سالباً");  
    }  
    $this->price = $price;  
}
```

- **__set()** منع:

php

```
public function __set(string $property, $value) {  
    if ($property === 'price') {  
        throw new Exception("setPrice لا يمكن تعديل السعر مباشرة. استخدم");  
    }  
}
```

مزايا التصميم:

1. إخفاء البيانات والتحقق من القيم: الأمان

2. سهولة إضافة ميزات جديدة: **المرونة**
3. تقسيم المسؤوليات بين الفئات: **الصيانة**
4. كل فئة يمكن اختبارها بشكل منفصل: **الاختبار**
- 5.